

Decomposing a Cloud

Master's Level Knowledge Assessment:
Architecture, Virtualisation & Serverless Execution

COM 769 | Scalable Advanced Software Solutions



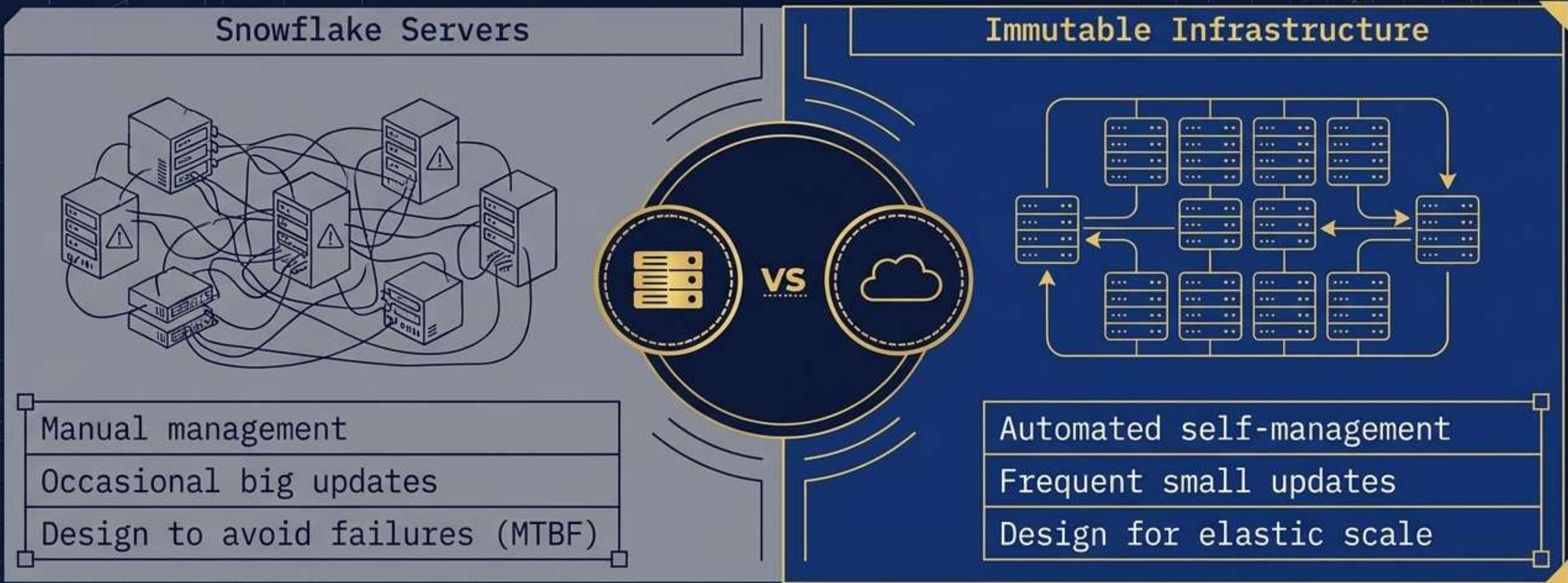
An architect notices that configurations on legacy on-premises servers drift over time due to manual updates, causing deployment failures and making environments impossible to replicate.

What modern cloud paradigm directly resolves this fragility?

- A) Serial and synchronised processing
- B) The intentional replication of Snowflake servers
- C) Immutable infrastructure
- D) Occasional, large-scale monolithic updates
- E) Strong eventual consistency

Correct Answer: C) Immutable infrastructure

Modern cloud environments favour servers that are rebuilt upon change rather than modified post-deployment. This eliminates Snowflake servers—unique, manually configured machines that cannot be easily reproduced.

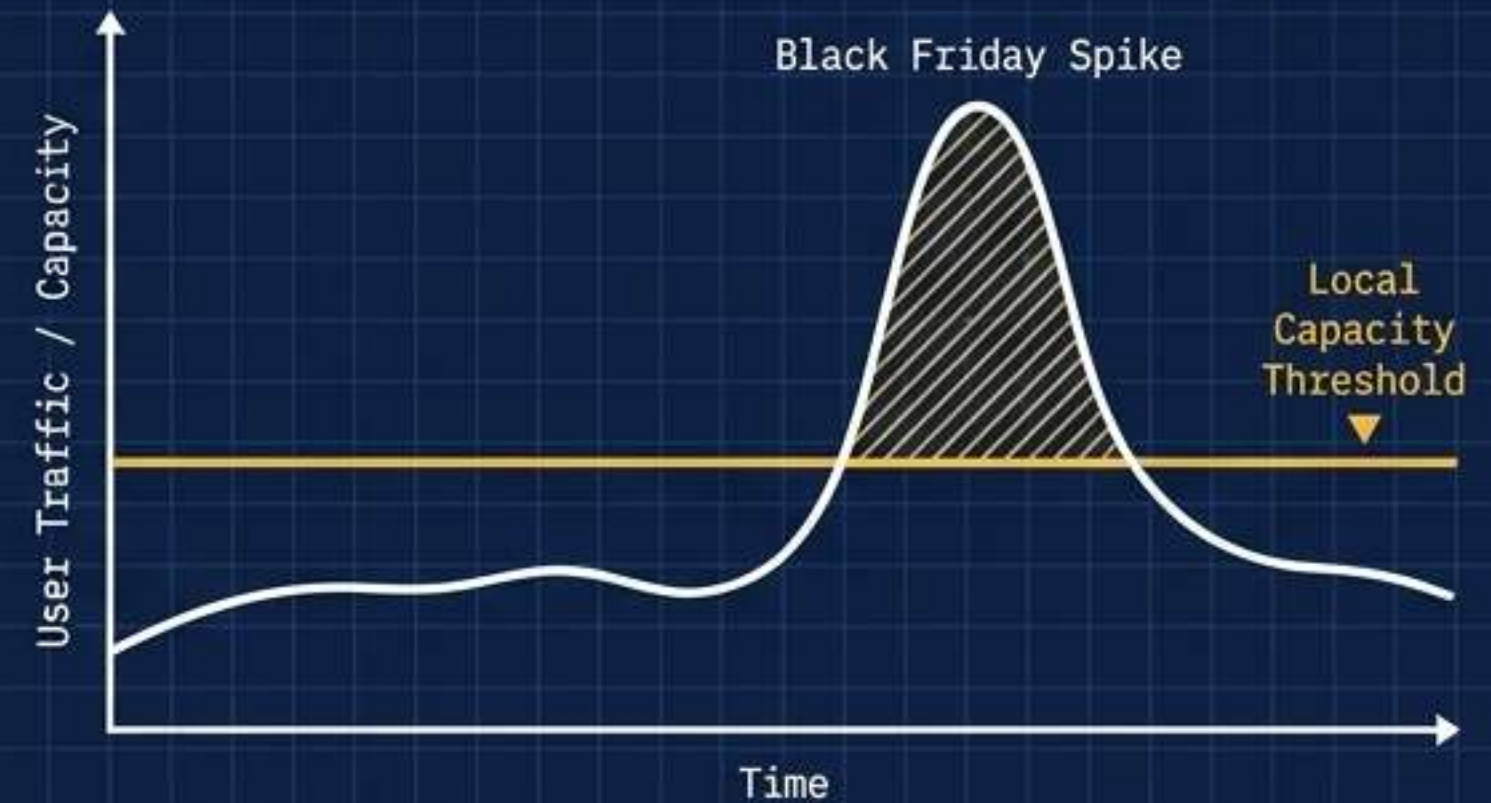


A retail organisation manages its core workloads on a private cloud to ensure compliance.

However, during the Black Friday sales event, user traffic vastly exceeds local capacity.

They require an architecture that dynamically provisions external resources only during this spike.

This mechanism is known as:



A) Polyglot persistence

B) Monolithic decentralisation

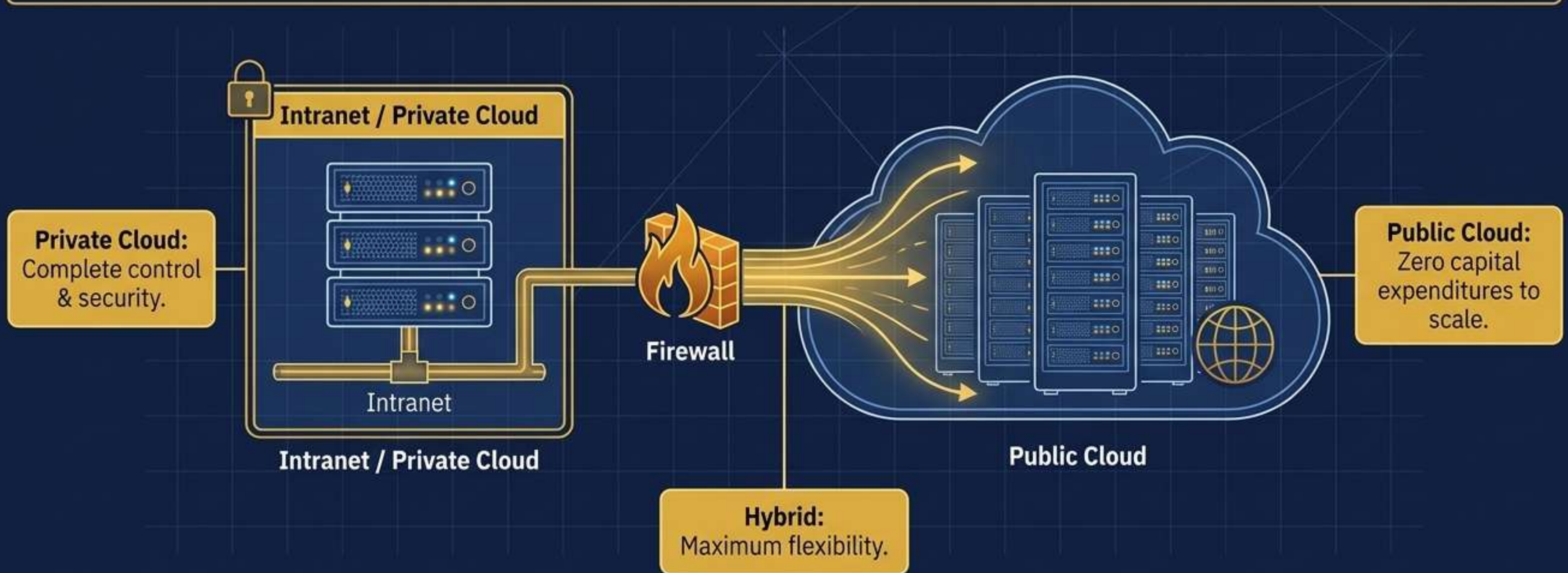
E) Automated hardware provisioning

C) Distributed eventual consistency

D) Hybrid cloud bursting

Correct Answer: **D) Hybrid cloud bursting**

A **hybrid cloud** combines owned private infrastructure with leased public cloud resources. Cloud bursting enables an organisation to use its private cloud for baseline needs, dynamically spilling over into the public cloud exclusively when utilisation exceeds private capacity.



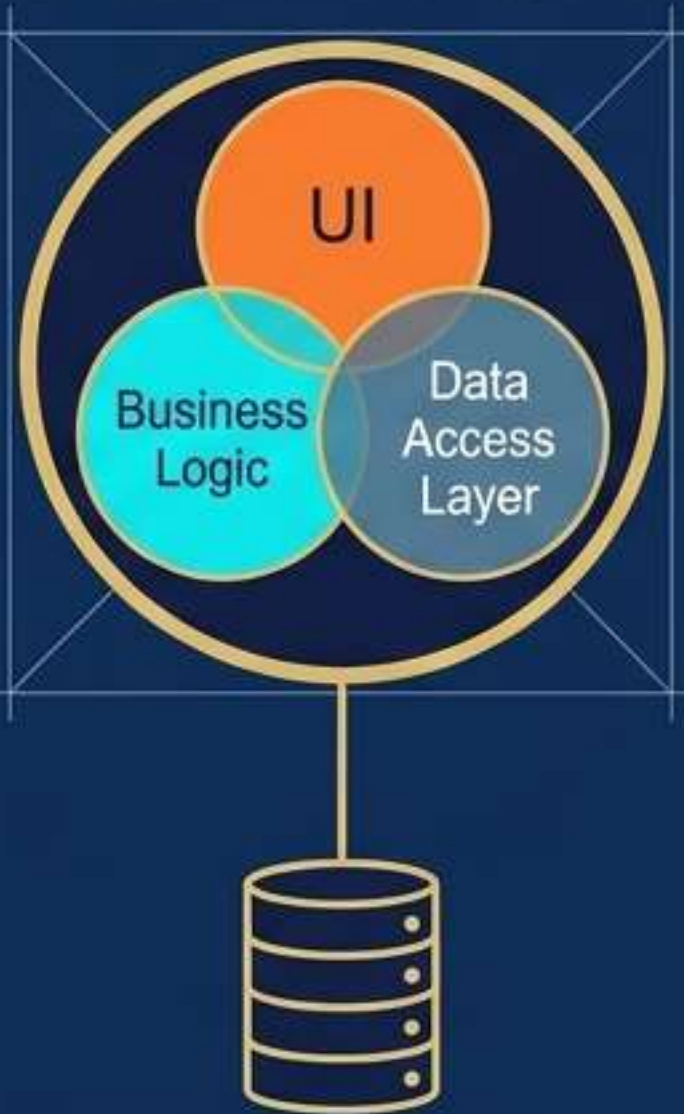
Decomposing a monolithic application into smaller, decentralised microservices provides horizontal scalability. However, this architectural shift introduces significant new complexities. Which of the following is a primary challenge inherent to microservice architectures?

- A) Forcing all operations to process serially and synchronously
- B) Managing application state that is now distributed across multiple services
- C) Maintaining a single, centralised relational database for all data access
- D) Ensuring hardware-level monolithic coupling
- E) Designing the system exclusively for predictable, static scale

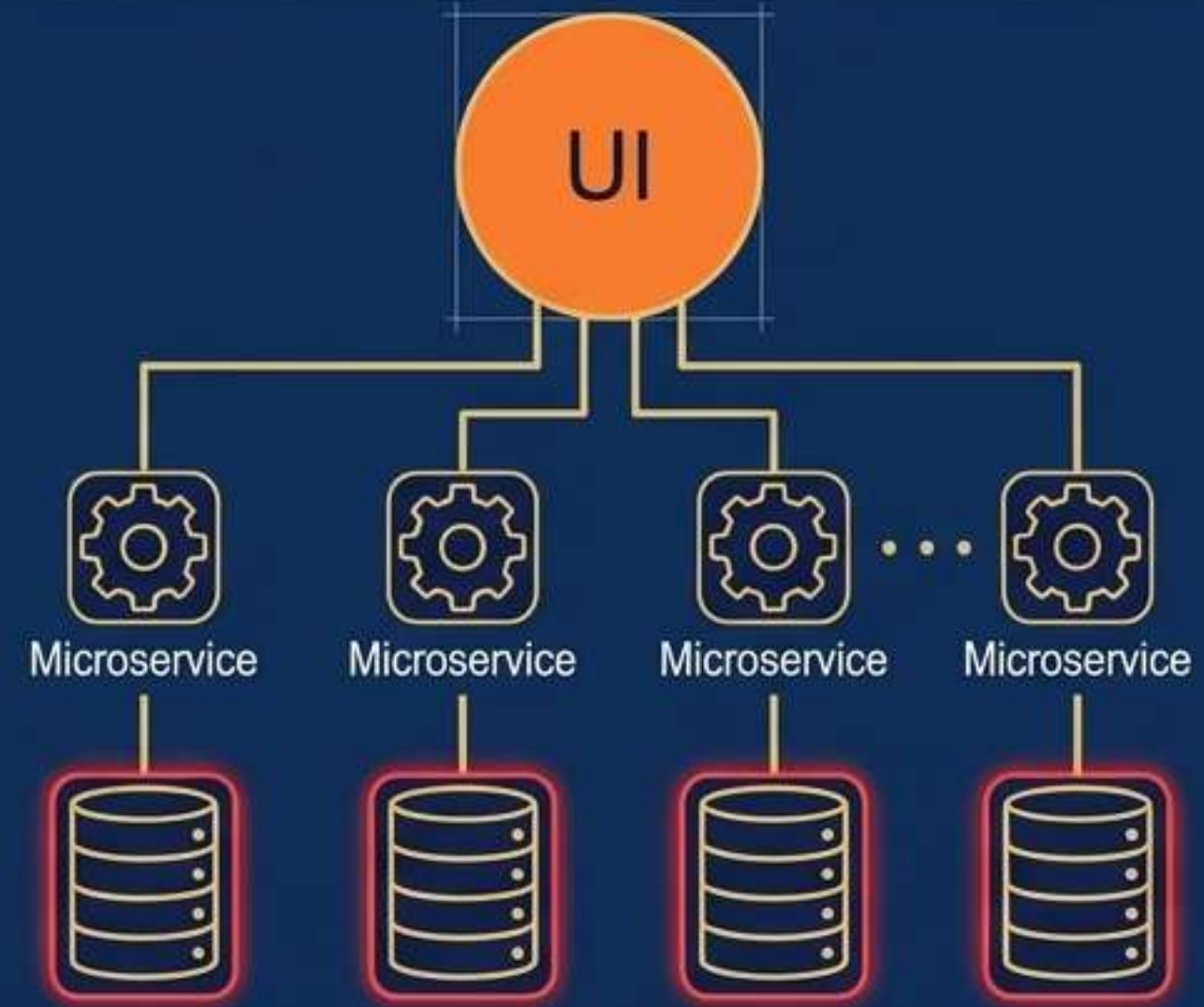
Correct Answer: B) Managing application state that is now distributed across multiple services

The cloud changes application design. Decentralised services communicate via APIs or asynchronous messaging. While this allows horizontal scaling, it means application state is no longer central—it is distributed, operations happen in parallel, and the system must be designed for resilience (MTTR) rather than just avoiding failure.

Monolithic Architecture



Microservice Architecture



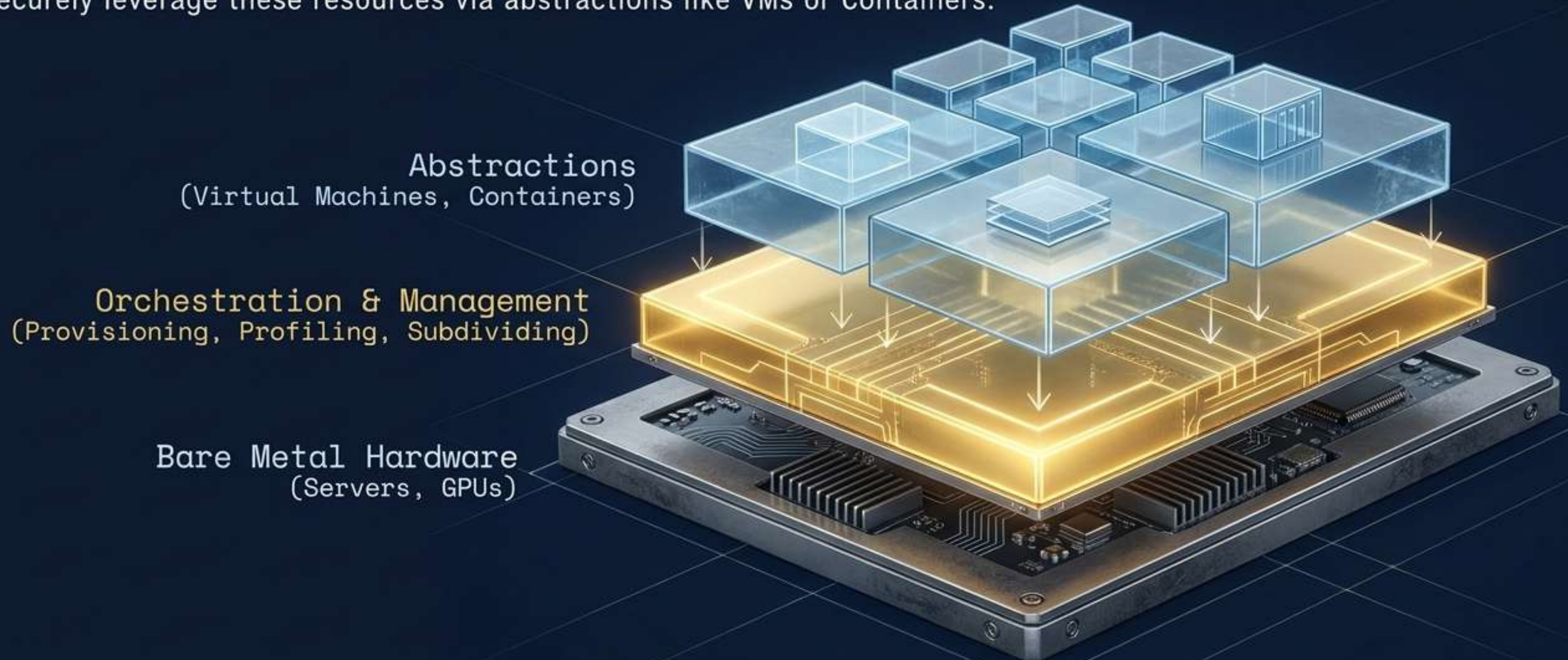
In a cloud datacentre, the physical bare metal must be abstracted into usable compute instances. What are the fundamental responsibilities of the Orchestration and Management layer?

- A) Rewriting application source code to ensure eventual consistency
- B) Managing the front-end user interface for monolithic software
- C) Provisioning, profiling, and subdividing the capabilities of bare hardware
- D) Managing the deployment of mobile application backends exclusively
- E) Defining the strong consistency rules for relational databases

Correct Answer:

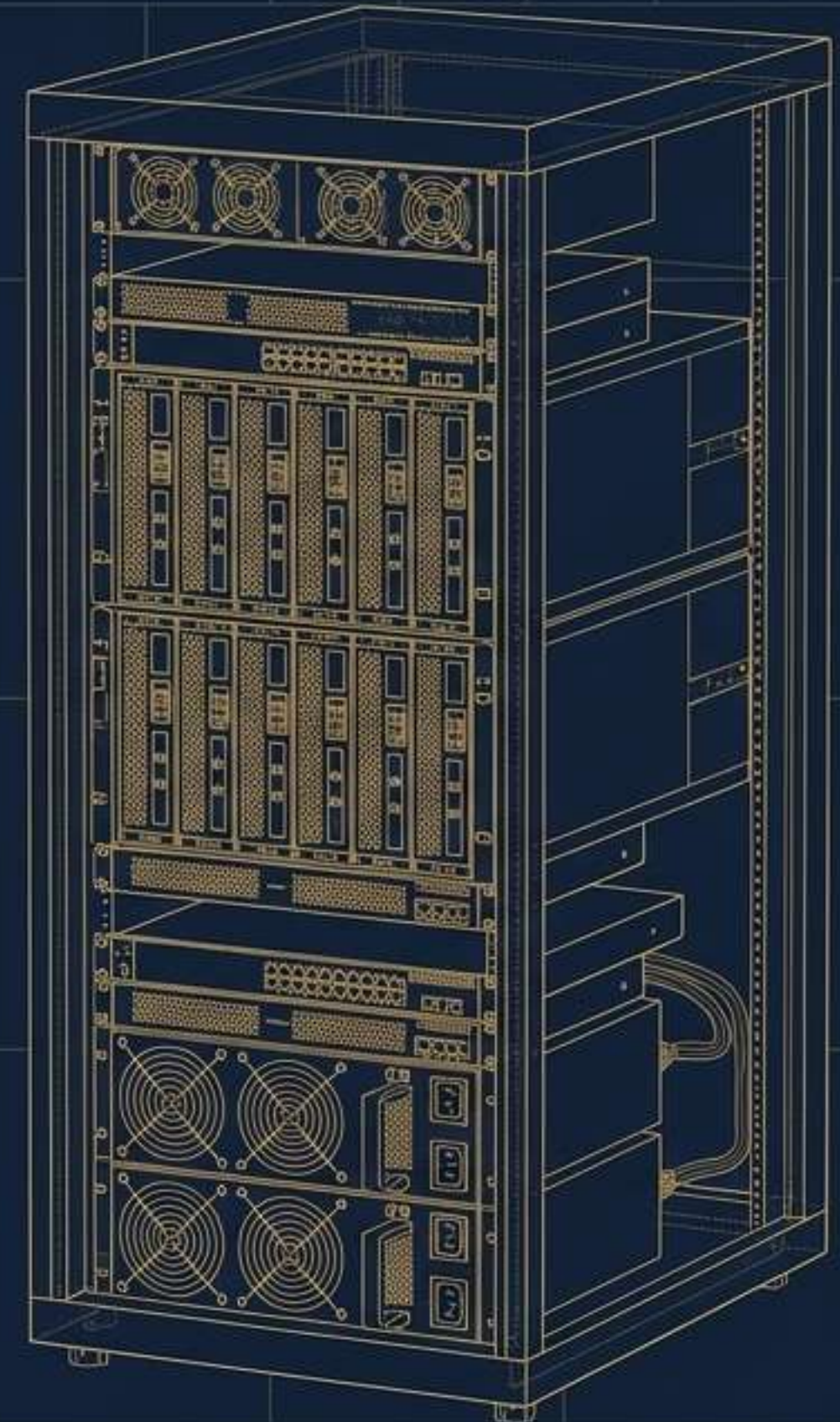
C) Provisioning, profiling, and subdividing the capabilities of bare hardware

Orchestration is the bridge between physical metal and **virtual resources**. It enrolls bare hardware into a management framework, profiles its capabilities (Compute, RAM, GPU), and provides the mechanisms to divide and securely leverage these resources via abstractions like VMs or Containers.



While lightweight containers are increasingly popular, Virtual Machines (IaaS) remain the optimal architectural choice under specific conditions. Under which scenario is deploying a Virtual Machine the most appropriate solution?

- A) When deploying a web app that requires micro-billing down to the millisecond
- B) When establishing a disaster recovery environment requiring total control over the operating system
- C) When abstracting away all server and infrastructure management is the primary goal
- D) When executing background tasks triggered exclusively by HTTP events
- E) When spinning up hundreds of isolated application instances that must share a single OS kernel



Correct Answer: B) When establishing a disaster recovery environment requiring total control over the operating system

Virtual Machines provide **Infrastructure as a Service (IaaS)**. Because they emulate physical computers entirely—including their own virtual processor, memory, and complete operating system—they are ideal for extending on-premises datacentres to the cloud or running critical apps during disaster recovery where total environmental control is mandatory.

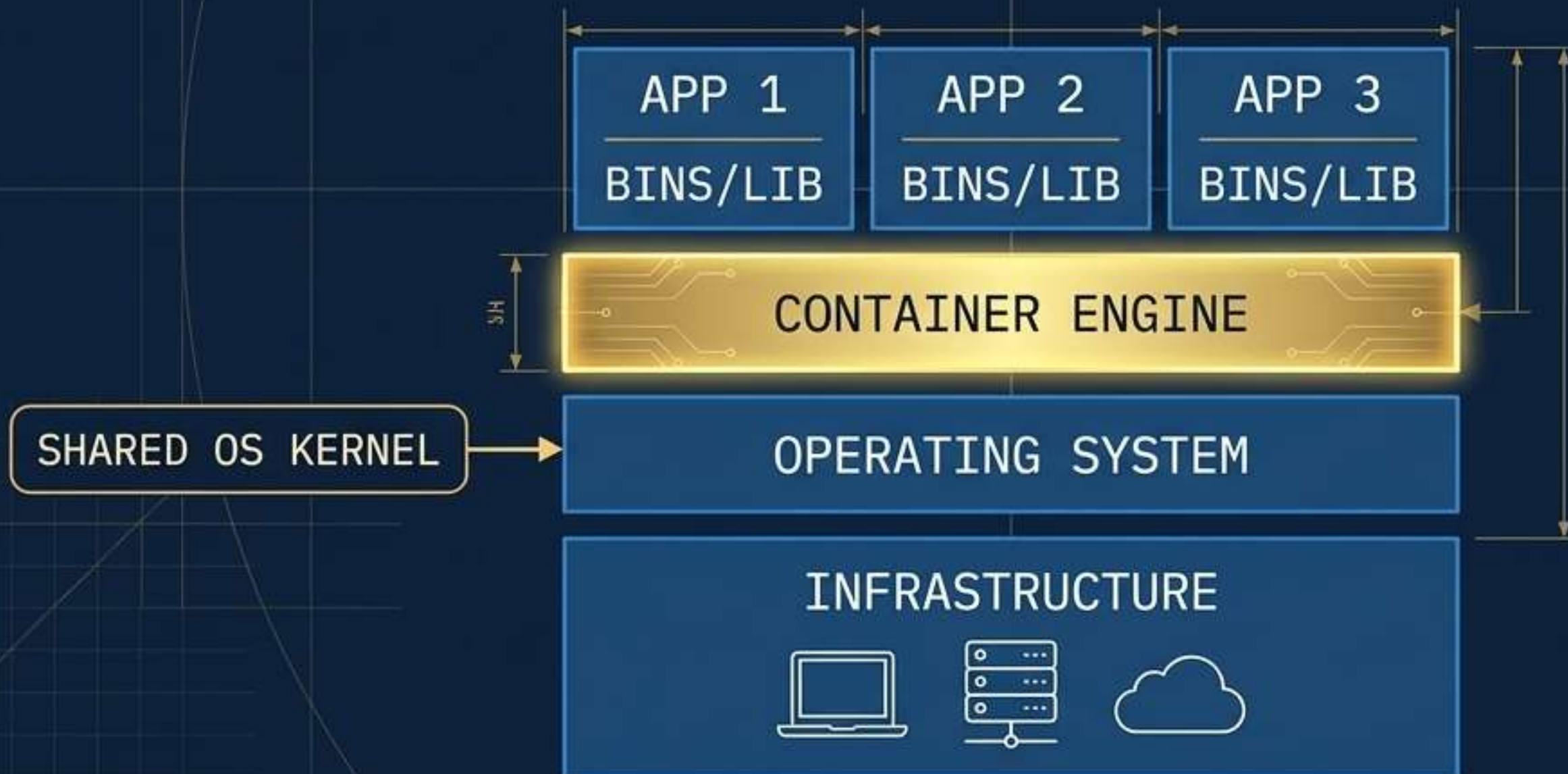


A development team is debating whether to package their microservices in Virtual Machines or Containers. What is the fundamental architectural distinction regarding how these two technologies interact with the operating system?

- A) Virtual Machines share the host OS kernel; Containers require a dedicated OS per instance
- B) Containers share the underlying host OS kernel; Virtual Machines host a separate, isolated OS per instance
- C) Containers run directly on bare metal hardware without any operating system abstraction
- D) Virtual Machines utilise a Container Engine, whereas Containers rely on a Hypervisor
- E) Neither technology utilises an operating system; both run purely on application binaries

Correct Answer: B) Containers share the underlying host OS kernel; Virtual Machines host a separate, isolated OS per instance

Virtual machines are limited to a single operating system per VM, making them resource-heavy. Containers are lightweight, virtualised application environments. Multiple containers run on a single physical/virtual host, maintaining strong segregation while sharing the single underlying OS Kernel.



An engineer needs to rapidly deploy a standalone Docker container to process a batch job. They want the fastest implementation possible without adopting orchestration services like Kubernetes or managing underlying virtual machines. Which PaaS offering is designed for this exact requirement?

A) Azure Durable Functions

B) Azure App Service Mobile Apps

C) Bare Metal Orchestration Clusters

D) Azure Container Instances (ACI)

E) On-Premises Monolithic Servers

Correct Answer: D) Azure Container Instances (ACI)

Standalone Container Instances offer the fastest and simplest way to run a container in the cloud. As a Platform as a Service (**PaaS**) offering, it removes the need to provision virtual machines or configure complex orchestration layers like **Kubernetes**. You simply upload the container, and the platform runs it.



The Cloud Architect's Blueprint

Serverless computing represents the highest level of cloud compute abstraction. Which three core characteristics fundamentally define the Serverless computing paradigm?

- A) Monolithic architectures, upfront capacity reservation, and strong relational consistency
- B) Hardware provisioning, operating system maintenance, and predictable static scaling
- C) Shared kernels, container orchestration, and manual deployment tracking
- D) The abstraction of servers, event-driven scaling, and micro-billing
- E) Decentralised monoliths, polyglot persistence, and serial processing

Correct Answer: D) The abstraction of servers, event-driven scaling, and micro-billing

In serverless architectures, the cloud platform manages all infrastructure and resource allocation automatically. You do not reserve capacity. Applications scale instantaneously based on events, and billing is calculated exclusively for the exact compute duration and memory used (micro-billing).

Abstraction



Infrastructure isn't your responsibility.

Event-Driven Scale



Scaling handled automatically based on demand.

Micro-Billing



Billed only for exact resources used, zero reservation.

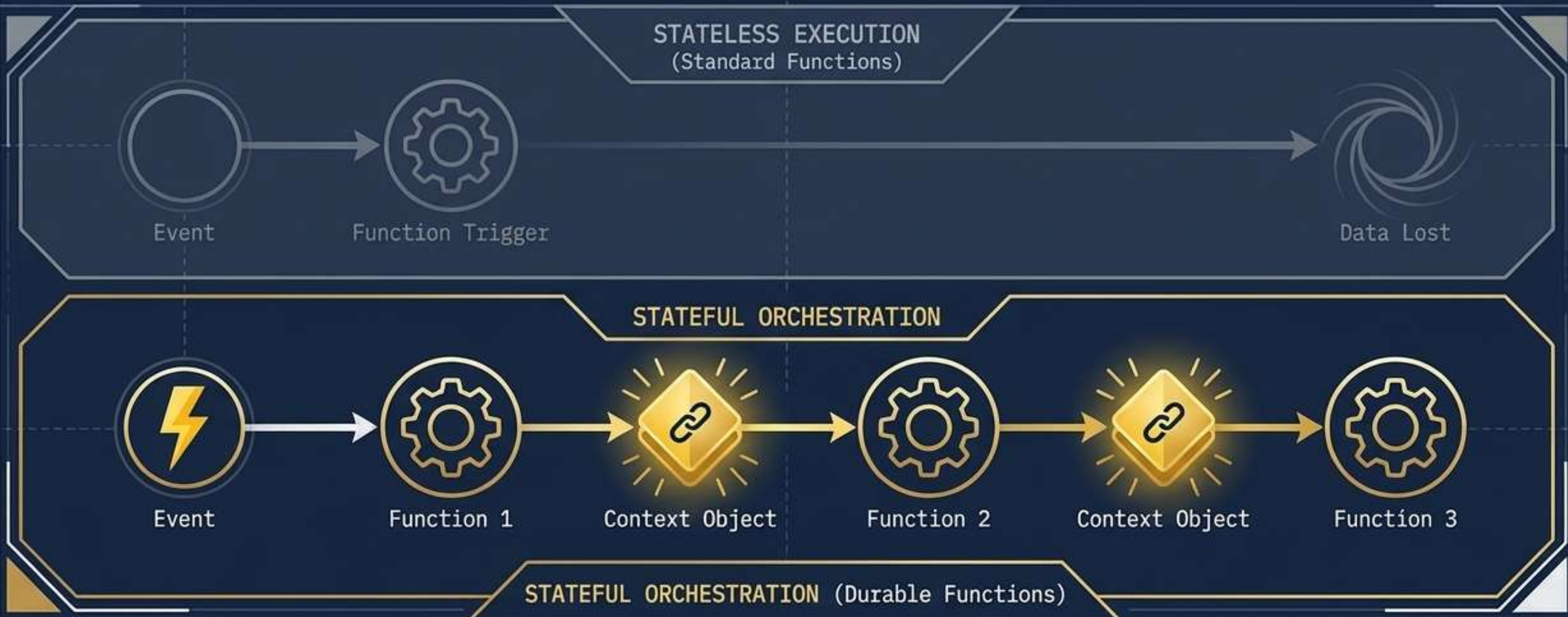
The Cloud Architect's Blueprint

Paradox: By default, Azure Functions are stateless; they behave as if restarted every time they respond to an event. However, complex workflows often require tracking prior activity. How is statefulness managed within this serverless environment?

- A) By storing session tokens directly on the bare metal hardware
- B) By provisioning a background WebJob to hold state continuously in memory
- C) By using Durable Functions to pass context through the function executions
- D) By reverting the architecture to an IaaS Virtual Machine
- E) By embedding state data within the Container Engine kernel

Correct Answer: C) By using Durable Functions to pass context through the function executions

While standard Functions are stateless, **Durable Functions** provide a mechanism for **stateful orchestration** in a serverless environment. They work by passing a **state context** through the function executions, allowing developers to track prior activity and chain complex workflows together without managing servers.



Context Object maintains state across functions, enabling complex workflows without servers.

The Cloud Architect's Blueprint

Synthesis Question: Cloud architecture requires balancing control against agility. Which sequence correctly orders the compute models from "Highest Management Overhead & Slowest Scale" to "Lowest Management Overhead & Fastest Scale"?



- A) Serverless -> Containers -> Virtual Machines
- B) Virtual Machines -> Serverless -> Containers
- C) Containers -> Virtual Machines -> Serverless
- D) Virtual Machines -> Containers -> Serverless
- E) Serverless -> Virtual Machines -> Containers

Correct Answer: D) Virtual Machines -> Containers -> Serverless

As we move up the abstraction layer cake, the burden of infrastructure management decreases, and the speed of scaling increases. VMs offer total control but slow boot times. Containers share a kernel for fast scaling but require orchestration. Serverless abstracts infrastructure entirely, offering instant scaling and micro-billing.

The Cloud Architect's Abstraction Matrix

Increasing Abstraction & Agility

Virtual Machines (IaaS)

- High Management (OS Updates)
- Slow Boot (Minutes)
- Scale Unit (Complete OS)

Containers (CaaS/PaaS)

- Medium Management (Orchestration)
- Fast Boot (Seconds)
- Scale Unit (Application + Bins/Libs)

Serverless (FaaS)

- Zero Infrastructure Management
- Instant Boot (Milliseconds)
- Scale Unit (Single Function/Event)

Increasing Control & Management Overhead